

Optimizing RLHF Training for Large Language Models with Stage Fusion

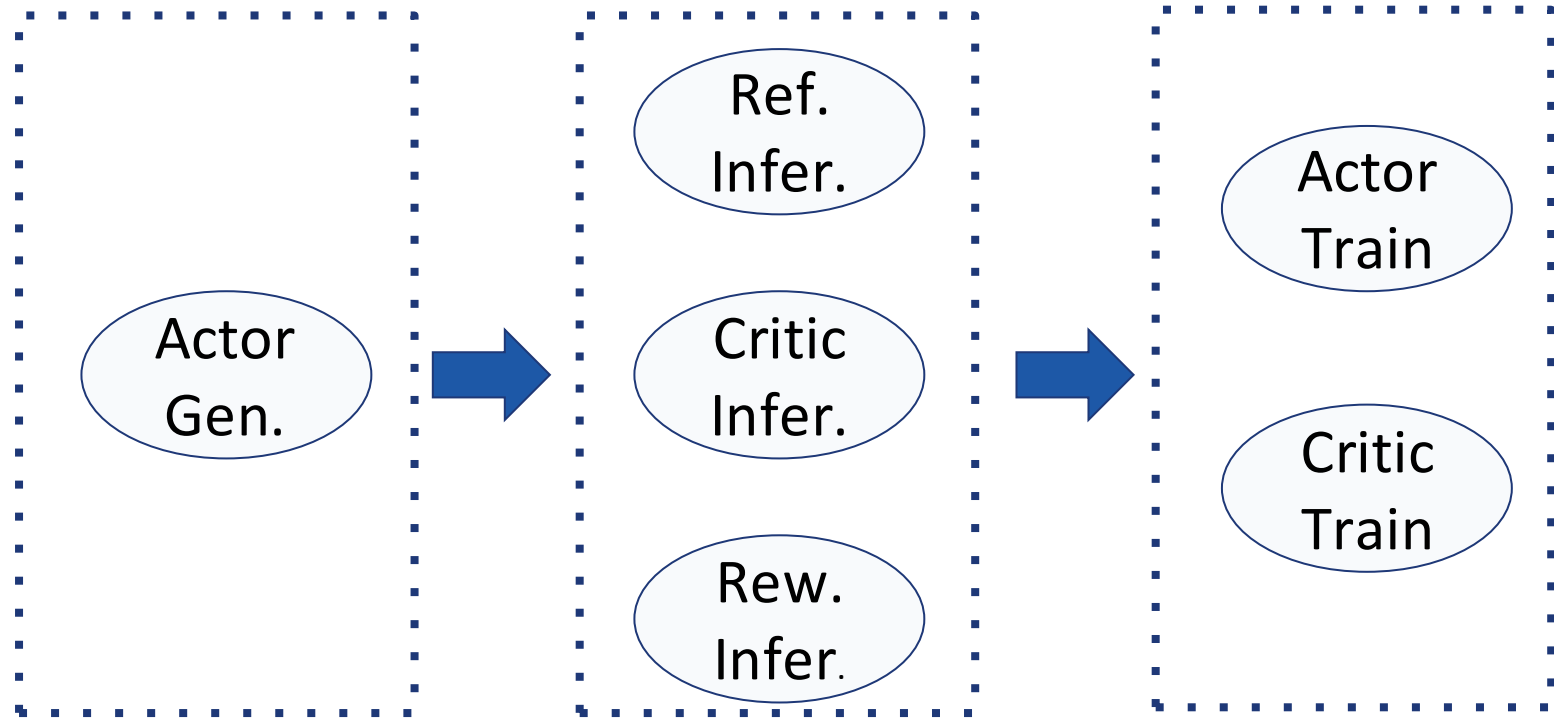


UNIVERSITY OF
ILLINOIS
URBANA-CHAMPAIGN

Agenda

1. RLHF Workflow
2. Motivation
3. Data-Aware Inter-Stage Fusion
4. Model-Aware Intra-Stage Fusion
5. Evaluation
6. Conclusion

RLHF Workflow

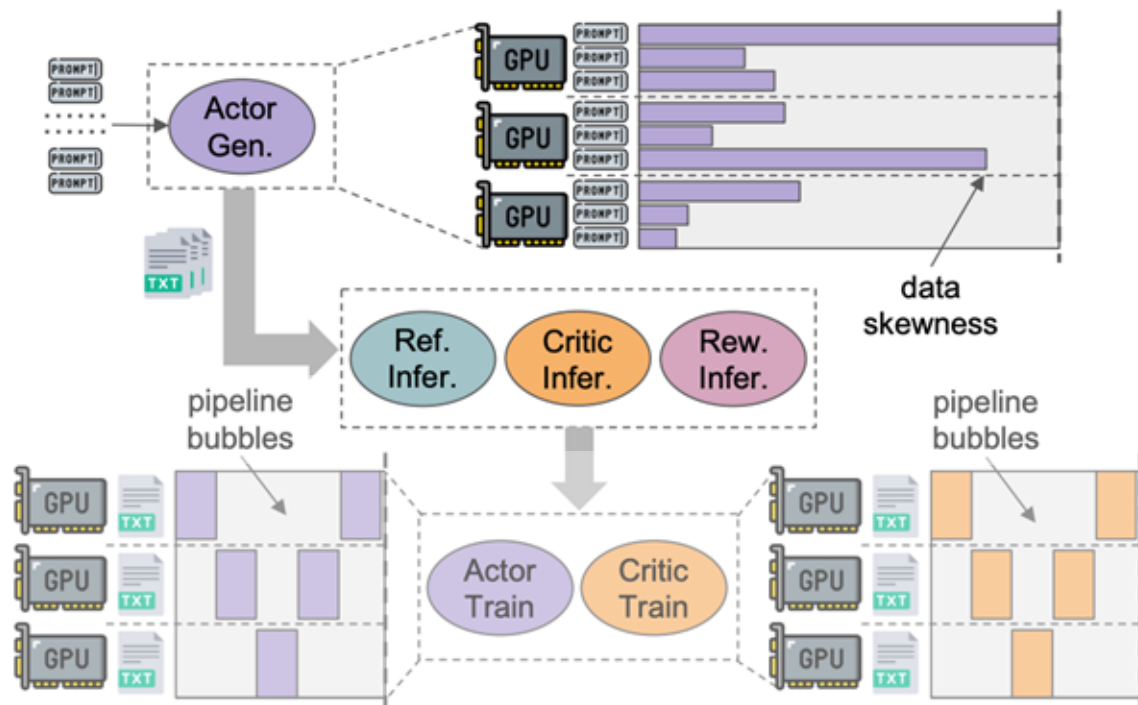


Problem: Data Skewness (Gen.)

Long-tailed samples
Data Dependencies

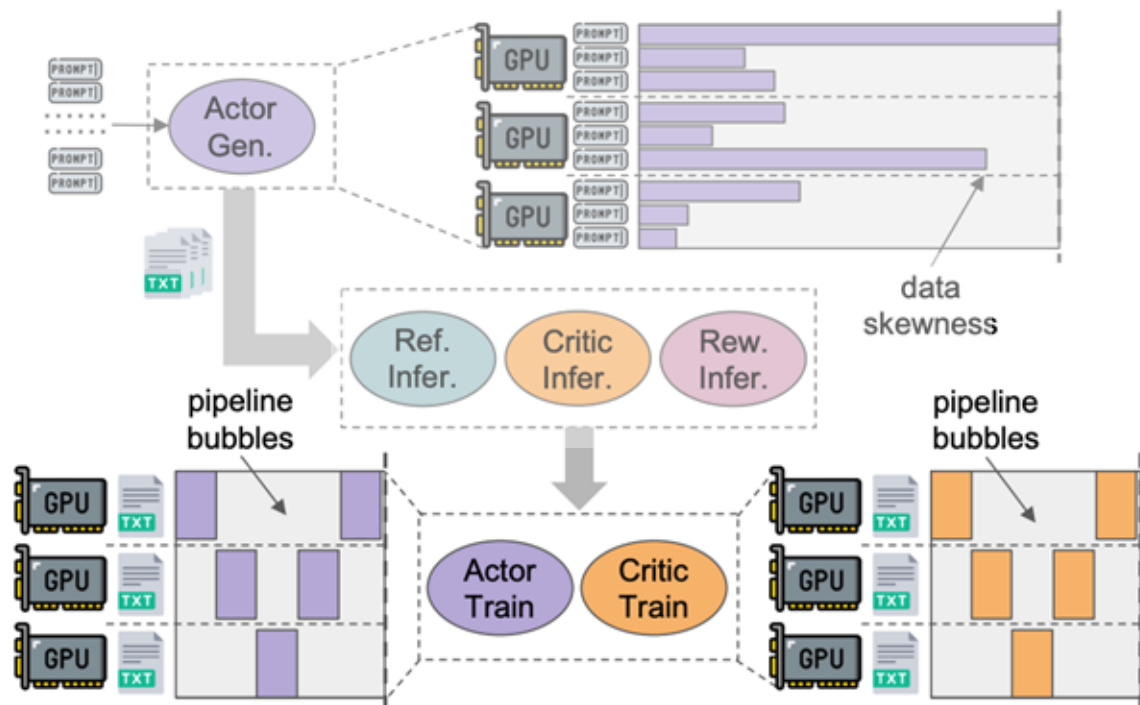


Low GPU Utilization



Problem: Pipeline Bubbles (Train)

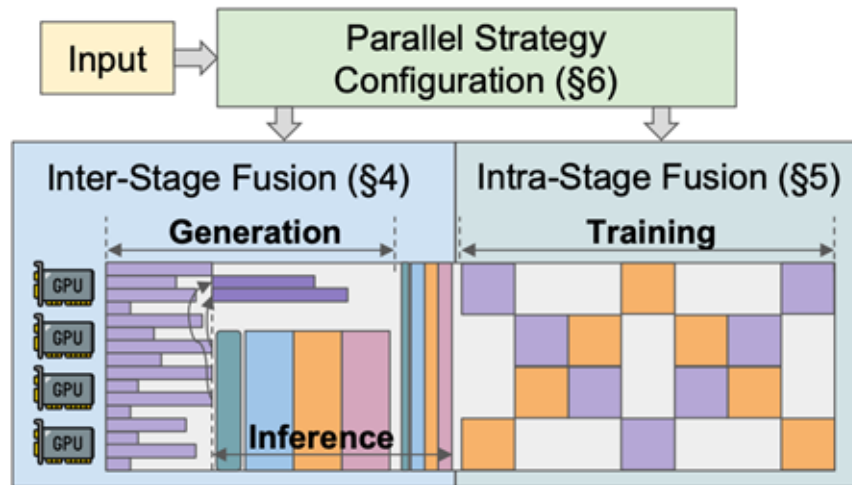
1. Scaling the PP size
2. 1F1B: Limited micro-batch size
3. Interleaved 1F1B: K-fold communcation



RLHFuse Overview

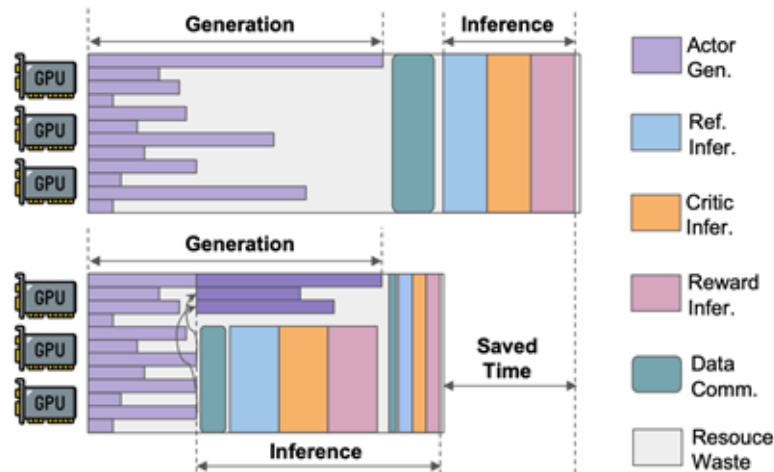
Inter-Stage Fusion: Migrates long-tail samples to overlap Generation with Inference

Intra-Stage Fusion: Fills bubbles by interleaving Actor and Critic tasks within the same stage



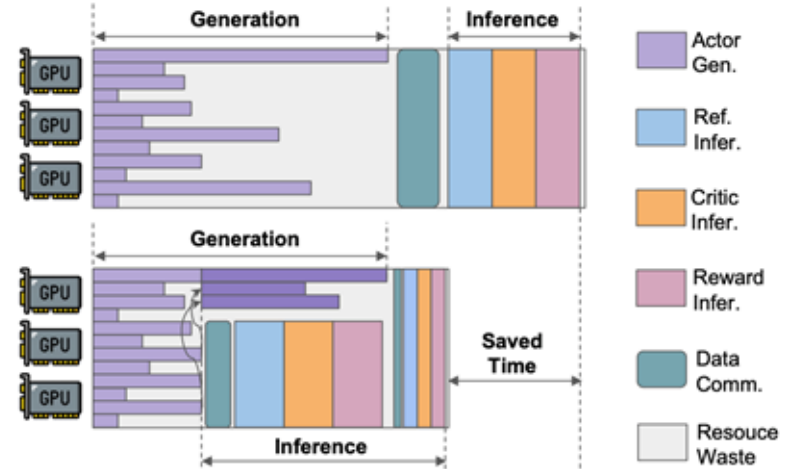
Data-Aware Inter-Stage Fusion

1. Triggering: Determines when to migrate to avoid high overhead or low overlap
2. Destination: Minimum GPUs to satisfy constraints
3. Mechanism: Migrates KV cache via RDMA to avoid re-prefilling



Migration Triggering

1. Trigger when the number of remaining samples in the generation stages is below R_t
1. Offline generation trials:
Sample length distribution
1. Simulation for R_t ranging from 5% - 95%



Migration Destination

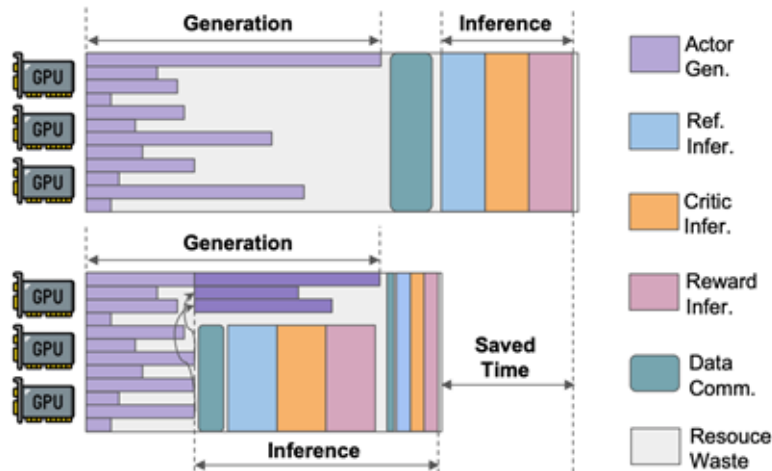
$$m \geq \max \left(\frac{R_t}{BS_{max}}, \frac{R_t * M}{C} \right)$$

Goal: Minimum m GPUs

Constraint 1: Per-GPU batch size

$\leq BS_{max}$

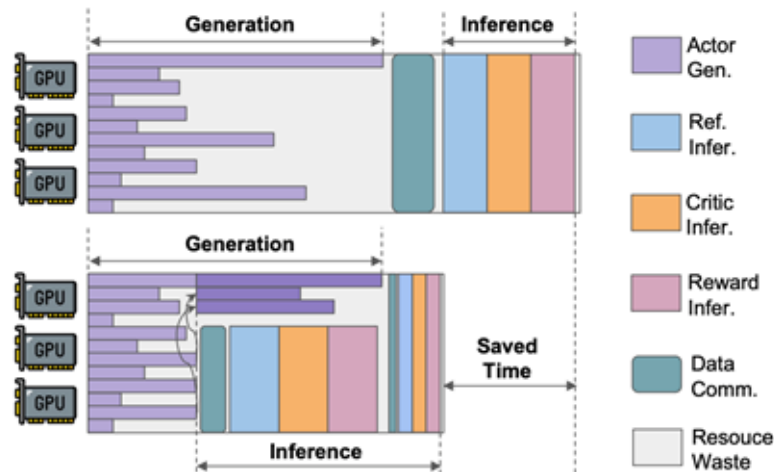
Constraint 2: Total KV-cache fits
in available memory C



Migration Mechanism

Trade-off: Transfer KV cache or
recompute KV cache (prefilling)

Author choice: Transfer KV cache
due to RDMA



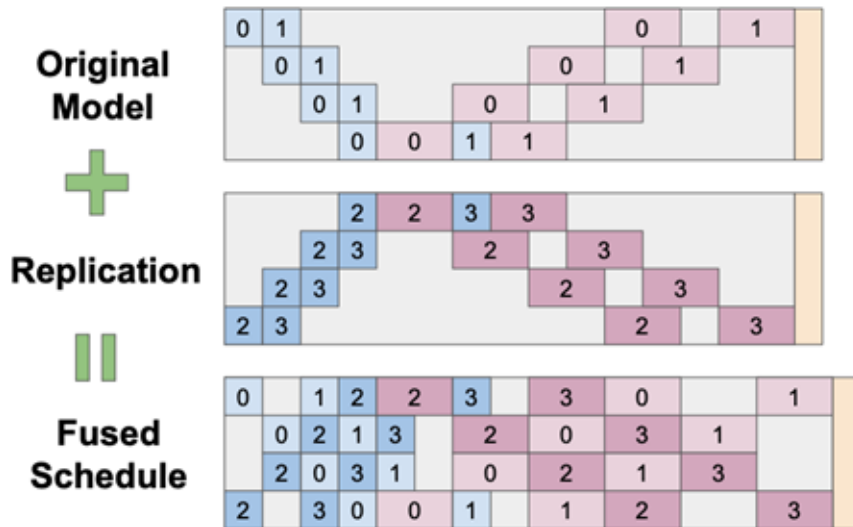
Model-Aware Intra-Stage Fusion

Fusion Training of Actor and Critic

→ fusing two pipeline in reversed order

Key Problems to solve

1. Heterogeneous model with different TP, PP
2. How to schedule the pipeline
3. Prevent out-of-memory

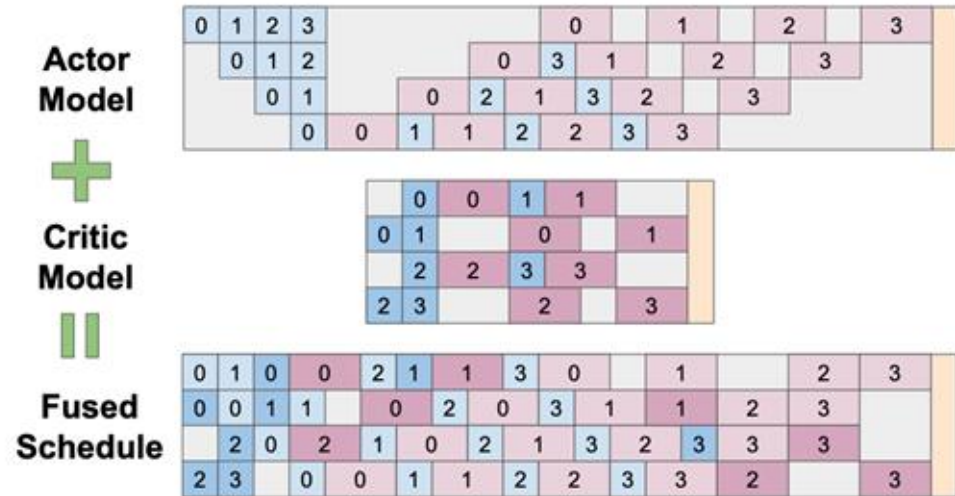


(a) Chimera [19]

Heterogeneous Actor and Critic

An example of heterogeneous model

- ❑ Different TP and PP
 - ❑ Hardware Misalignment
 - ❑ Higher Scheduling Complexity
- ❑ Other problems:
 - ❑ Memory Bottleneck
 - ❑ Complex Dependency across nodes



(b) RLHFuse

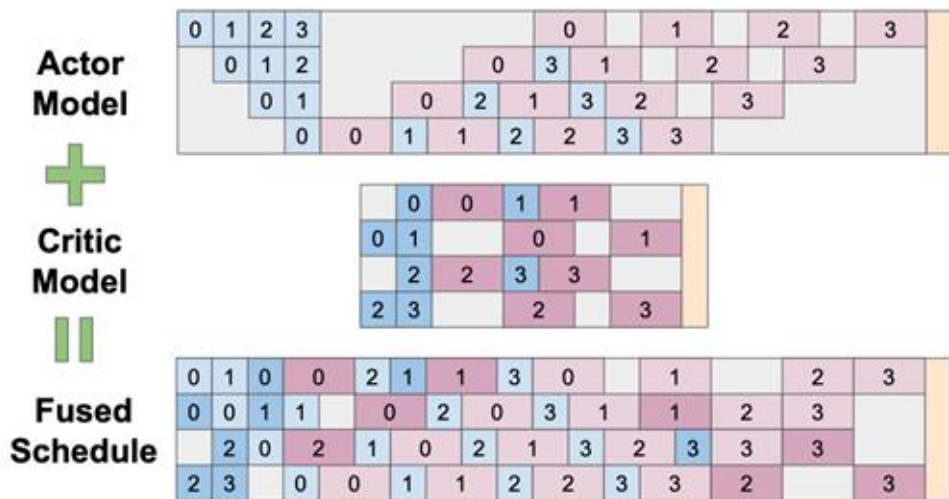
Heterogeneous Actor and Critic

TP Alignment

- Assume $TP1 = s \times TP2$
 - Merge s pipeline stages in model 2
 - PP2 must be divisible by s
 - Now each stage in both model use same number of GPUs

PP Alignment

- Let $N = \text{LCM}(N1, N2)$
 - $K1 = N/N1, K2 = N/N2$
 - Since global batch size is fixed:
 - $K1 \times M1 = K2 \times M2 = M$
 - Batch Size M divisible by $K1, K2$.



(b) RLHFuse

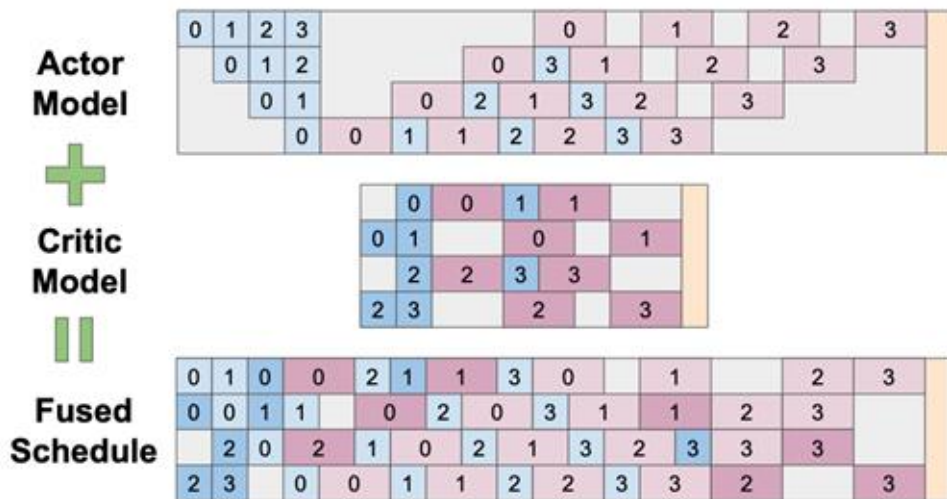
Fused Pipeline Schedule

Problem Formulation

- ❑ There are N stages
- ❑ Each micro-batch is processed exactly once
- ❑ $N \times 2$ ($M1 + M2$) subtasks to order
- ❑ Constrains:
 - ❑ Causal Dependency
 - ❑ Memory Limit

Algorithms

- ❑ **Greedy**
 - ❑ Lack of global perspective
- ❑ **Simulated Annealing**
 - ❑ Acceptable local optimums



(b) RLHFuse

Simulated Annealing

The Scheduling Algorithm

- ❑ Greedy solution as starting point
- ❑ Randomly swap consecutive subtasks to find a neighboring schedule.
- ❑ Estimated elapse time as energy
- ❑ Accept worse solution with probability based on temperature
- ❑ Decrease temperature through time

Algorithm 1 Generate Fused Pipeline Schedule.

```
1: function GENERATEFUSEDPIPELINESCHEDULE( $S_0$ )
2:    $s_{current} \leftarrow S_0$ 
3:    $e_{current} \leftarrow \text{ComputeEnergy}(s)$ 
4:    $T \leftarrow e_{current}$ 
5:    $s^* \leftarrow s_{current}$ 
6:    $e^* \leftarrow e_{current}$ 
7:   while  $T > \epsilon$  do
8:      $s_{neighbor} \leftarrow \text{ComputeNeighbor}(s_{current})$ 
9:      $e_{neighbor} \leftarrow \text{ComputeEnergy}(s_{neighbor})$ 
10:    if  $e_{neighbor} < e^*$  then
11:       $s^* \leftarrow s_{neighbor}$ 
12:       $e^* \leftarrow e_{neighbor}$ 
13:    if  $P(e_{current}, e_{neighbor}, T) > \text{Rand}(0, 1)$  then
14:       $s_{current} \leftarrow s_{neighbor}$ 
15:       $e_{current} \leftarrow e_{neighbor}$ 
16:     $T \leftarrow T \times \alpha$ 
17:  return  $s^*$ 
```

Simulated Annealing

Components

- ❑ **How to get a neighbor**
 - ❑ Randomly Swap Two adjacent subtask to get a new neighbor.
 - ❑ Validate the causal order of the new schedule.
- ❑ **How to compute energy**
 - ❑ Use the maximum elapsed time across all nodes as energy
 - ❑ Using a dynamic programming method to compute elapsed time.

Algorithm 2 Generate A Random Neighbor Schedule.

```
1: function COMPUTENEIGHBOR( $S$ )
2:   while True do
3:      $i \leftarrow \text{RandInt}(1, N)$ 
4:      $j \leftarrow \text{RandInt}(1, 2(M_1 + M_2) - 1)$ 
5:      $S' = \text{Swap}(S_{ij}, S_{i(j+1)})$ 
6:     if CheckValid( $S'$ ) then
7:       return  $S'$ 
```

Simulated Annealing

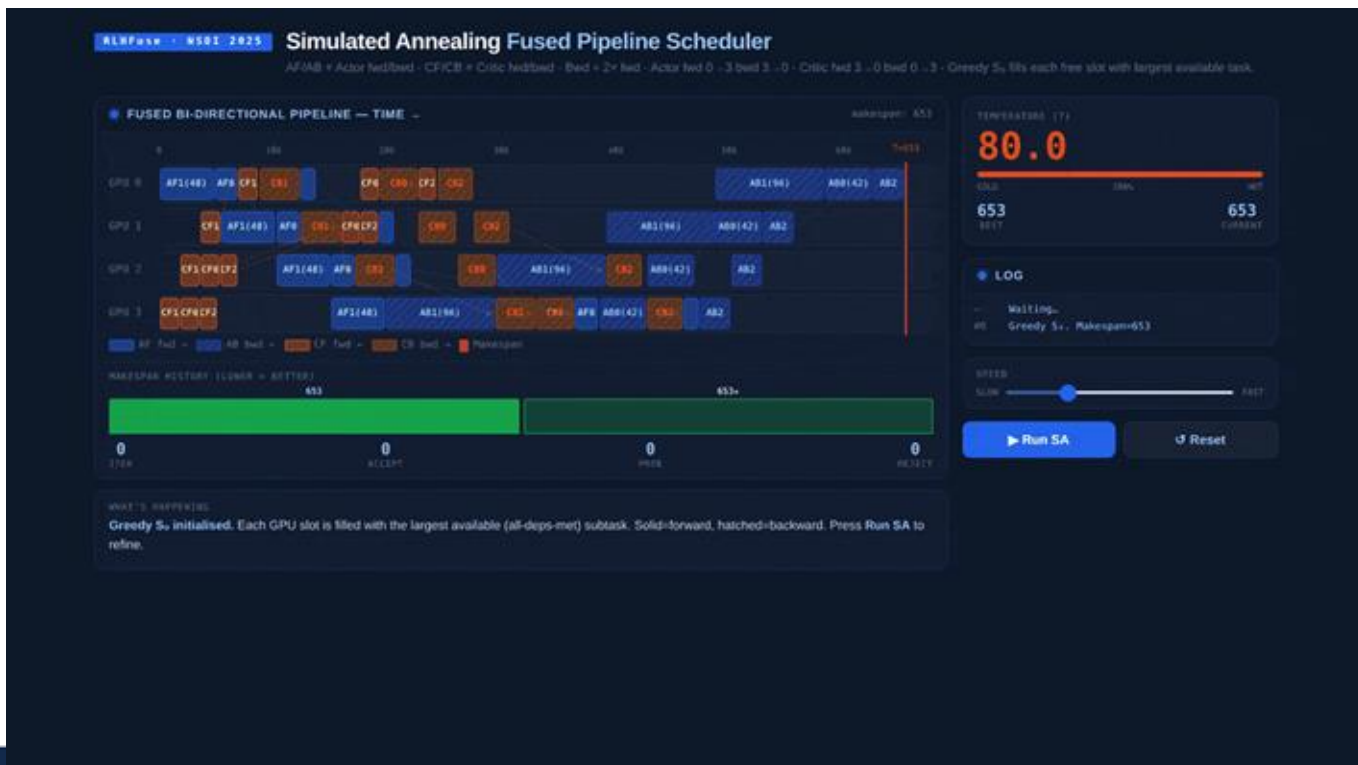
Components

- ❑ **How to get a neighbor**
 - ❑ Randomly Swap Two adjacent subtask to get a new neighbor.
 - ❑ Validate the causal order of the new schedule.
- ❑ **How to compute energy**
 - ❑ Use the maximum elapsed time across all nodes as energy
 - ❑ Using a dynamic programming method to compute elapsed time.

Algorithm 3 Compute Energy

```
1: function COMPUTEENERGY( $S$ )
2:   return  $\max_{1 \leq i \leq N} \text{ComputeFinishTime}(S_{i,M})$ 
3: function COMPUTEFINISHTIME( $S_{ij}$ )
4:   if  $S_{ij} \in \text{Memo}$  then return  $\text{Memo.get}(S_{ij})$ 
5:    $S^{intra} \leftarrow \text{GetIntraDependency}(S_{ij})$ 
6:    $S^{inter} \leftarrow \text{GetInterDependency}(S_{ij})$ 
7:    $\text{IntraTime} \leftarrow \text{ComputeFinishTime}(S^{intra})$ 
8:    $\text{InterTime} \leftarrow \text{ComputeFinishTime}(S^{inter})$ 
9:    $\text{FinishTime} = \max(\text{IntraTime}, \text{InterTime}) + l_{ij}$ 
10:   $\text{Memo.put}(S_{ij}, \text{FinishTime})$ 
11:  return  $\text{FinishTime}$ 
```

Simulated Annealing



Evaluation Settings

❑ Cluster Setting

- 32 nodes with 256 GPUs. Each node with 2TB RAM
- Nodes are connected with 8 x 200 Gbps RDMA network (InfiniBand)

❑ Dataset

- HH-RLHF (Helpful & Harmless RLHF)

❑ Model Configurations (LLaMA)

- Actor/Critic Size
 - 13B/33B, 33B/13B, 33B/65B, 65B/13B
- Batch Size:
 - 512
- Gen Length
 - 512, 1024, 2048

Baselines and System Optimization

❑ Baseline Frameworks

- DeepSpeed-Chat: All models share a single GPU pool using ZeRO-3 data parallelism only.
- ReaLHF: Optimizes parallel strategy for each model independently.
- RLHFuse-Base: Include all optimization in RLHFuse without Inter and Intra stage fusion.

❑ System Optimization

- Continuous Batching
 - vLLM-style continuous batching. Completed samples are immediately moved out.
- Faster GAE computation
 - Generalized Advantage Estimation is parallelized across GPUs.
- Adaptive Load Balancing
 - Dynamically redistributes micro-batches to busy GPUs, balancing compute load
- Task-Tailored 3D Parallelism
 - Each task (Actor, Generation, Reference, Inference) gets its own optimal TP/PP/DP

End-to-End Results

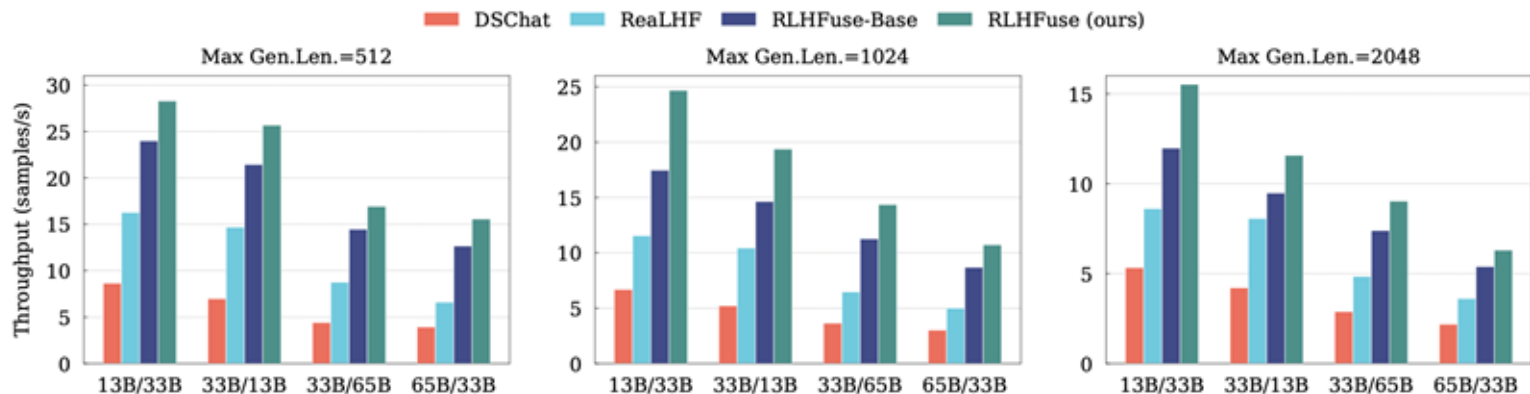


Figure 7: End-to-end throughput of RLHF training systems under different generation and model size settings.

3.7×

max improvement vs. DSChat

2.4×

max improvement vs. RealLHF

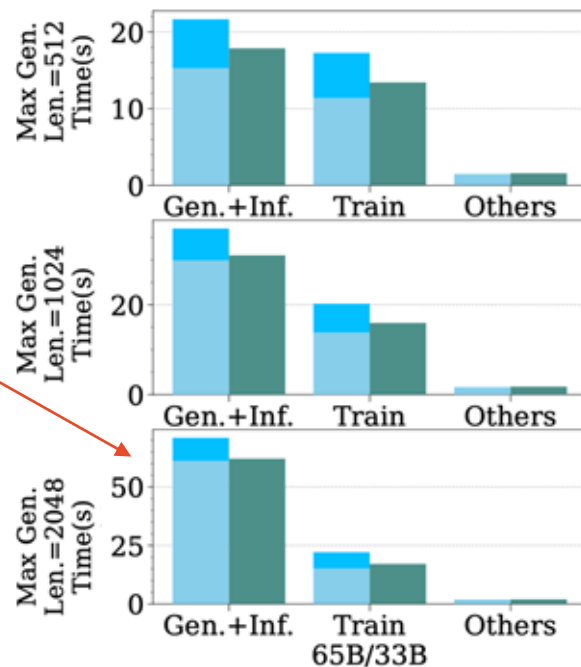
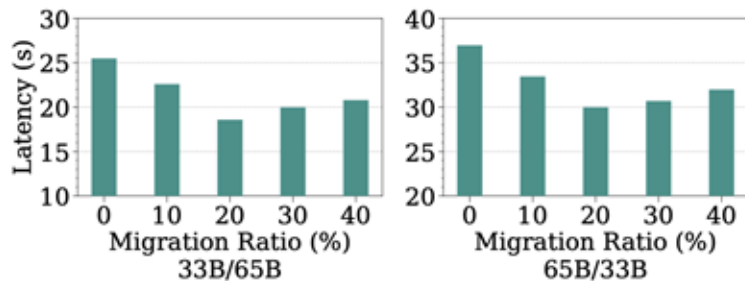
1.4×

improvement vs. RLHFuse-Base

Performance Breakdown

Generation + Inference Stage

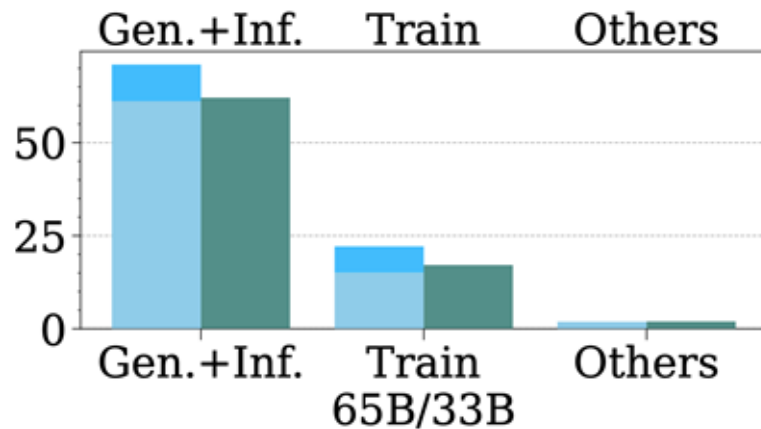
- ❑ Integer-Stage fusion reduce elapsed time by 20% to 40% across configurations.
- ❑ Effect is strongest for longer generation.
- ❑ Tuning on Migration rate might be needed



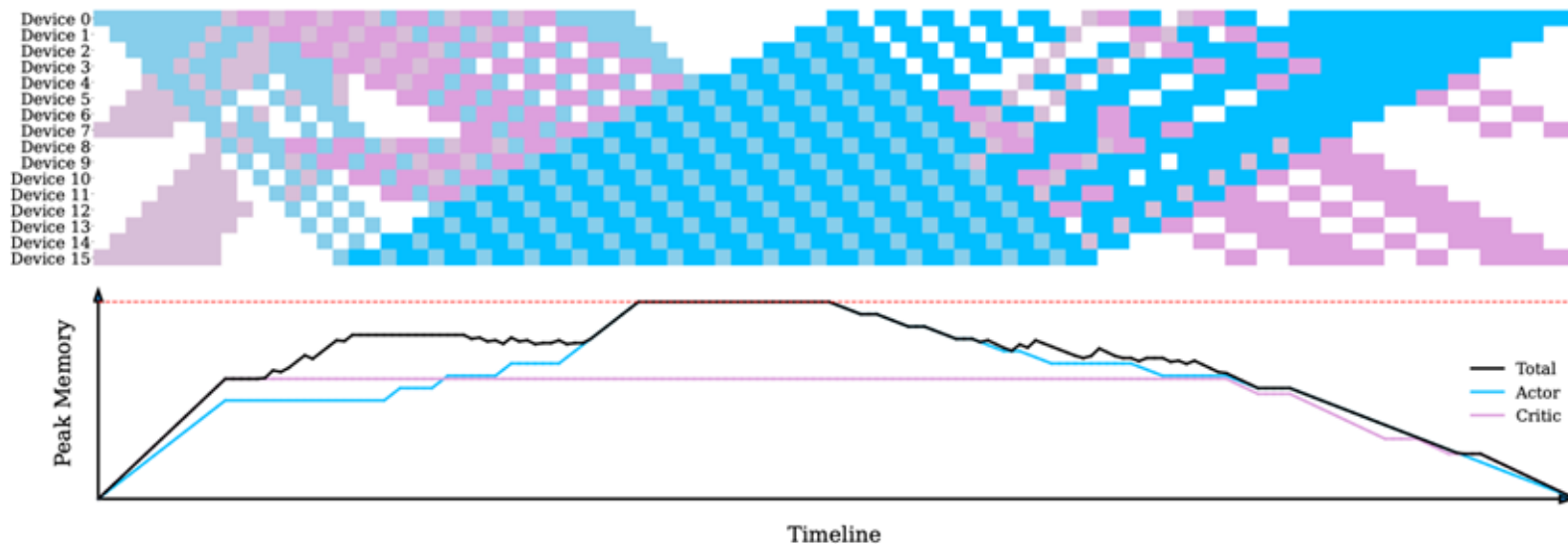
Performance Breakdown

Training Stage

- ❑ Intra-Stage fusion reduce training time by 15% to 25%.
- ❑ Scheduling algorithm reaches the theoretical lower bound in nearly configs.
- ❑ For 65B/33B setting: Entire critic training is hidden by Actor's pipeline bubbles.



Performance Analysis



The pipeline schedule of 65B Actor and 33B Critic

Strength & Limitations

✓ Strengths

Principled problem decomposition

Breaking at subtask level is a generalizable insight beyond RLHF — applicable to any multi-stage LLM workflow.

Near-optimal scheduling

Simulated annealing hits the theoretical lower bound on latency in nearly all configurations, with memory optimization as a secondary pass.

"Free" Critic training (65B/33B)

In the best case, 33B Critic training is entirely hidden inside the 65B Actor pipeline bubbles — zero added time.

Production-grade implementation

7K lines built on Megatron-LM; deployed internally at StepFun with real workloads.

✗ Limitations

High-bandwidth network dependency

KV-cache migration relies on fast RDMA (8×200Gbps). On lower-bandwidth networks the transfer cost may outweigh overlap savings.

Micro-bubble residuals from merging

When merging Critic pipeline stages to align TP, internal micro-bubbles are created that the scheduler cannot eliminate.

Homogeneous hardware assumption

The formulation assumes all GPUs have equal compute and memory. Heterogeneous clusters (cloud spot instances, mixed GPU types) are not handled.

Adapts to RLHF/PPO specifically

Inference ↔ training synchronization barrier exists for other RL algorithms too; applicability to async RL (e.g., GRPO) requires further work.

RLBoost: Harvesting Preemptible Resources for Cost-Efficient Reinforcement Learning on LLMs

Yongji Wu¹, Xueshen Liu^{2,3}, Haizhong Zheng⁴, Juncheng Gu³, Beidi Chen⁴,
Z. Morley Mao³, Arvind Krishnamurthy^{3,5}, Ion Stoica¹

¹UC Berkeley, ²University of Michigan, ³Google, ⁴CMU, ⁵University of Washington



UNIVERSITY OF
ILLINOIS
URBANA-CHAMPAIGN

Problem Definition | Synchronous RL Bottleneck

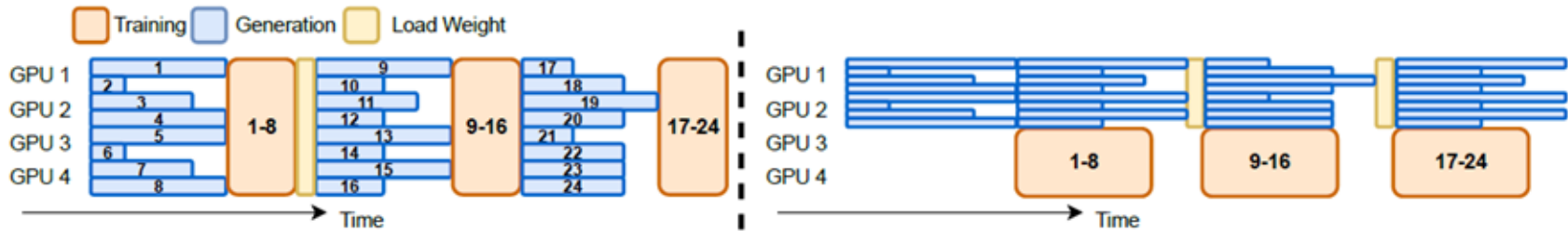
❑ Key Challenges

- Generation and training strictly alternate
- Batch update waits for longest rollout
- LRMs generate highly variable reasoning lengths
- Non-uniform decoding across GPUs
- Severe hardware underutilization

❑ Why It Matters

- LRMs produce long chain-of-thought reasoning
- Thousands of tokens per trajectory
- Large batch exploration required for PPO stability

- Time →
GPU1 : XXXXXXXXXX DONE
GPU2 : XXXXXXXXXX DONE
GPU3 : XXXXXXXXXXXXXXXXXXXXXXXXXXXX LONG OUTPUT
GPU4 : XXXXXXXXXX DONE
- The longest pole problem



System–Algorithm Conflict in Large-Scale RL

❑ Infrastructure Challenges

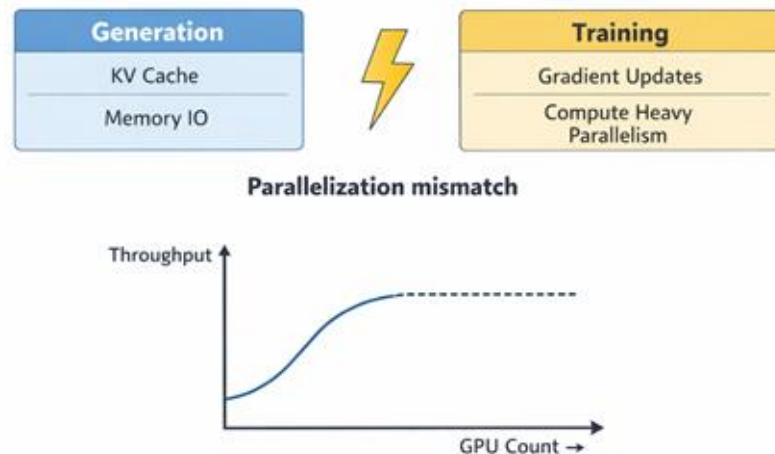
- Auto-regressive decoding → memory IO bound
- Heavy KV cache management
- Dynamic output lengths

❑ Training Requirements

- Compute-bound optimization
- Tensor + pipeline parallelism
- Large global batches

❑ Algorithm Constraints

- PPO requires on-policy samples
- Async rollout introduces staleness
- Policy mismatch degrades learning



Related Work: Algorithms Enabling Asynchronous RL

❑ Rule-Based RL for LRMs

- Verifiable rewards (math, coding)
- Long reasoning trajectories
- Replace preference-based RLHF

❑ Asynchronous RLH

- Idea: Separate rollout and training workers.
- Benefit: Higher utilization.
- Limitation: PPO sensitive to stale trajectories.

❑ Tapered Off-Policy REINFORCE (TOPR)

- Key Ideas: Importance sampling correction.
- Preserve high-reward trajectories.
- Reduce variance from negatives.
- Result: Stable off-policy learning.

Related Work: Algorithms Enabling Asynchronous RL

Framework	Approach	Limitation
veRL	HybridFlow + resharding	Switching overhead
OpenRLHF	Unified inference + training agents	Batch synchronization remains
PUZZLE	GPU time-sharing	Sequential dependency persists
StreamRL	Streaming rollout scheduling	Auxiliary predictor required

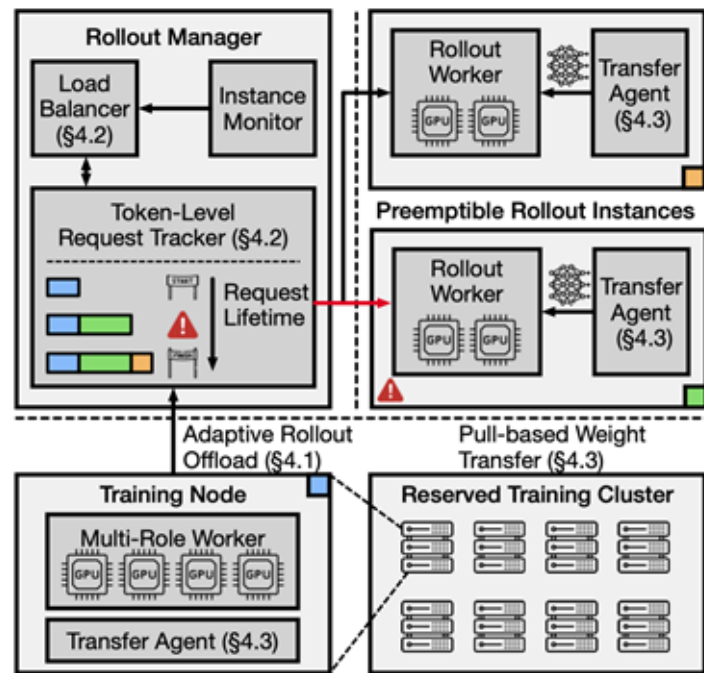
Method | RLBoost

Hardware Instance

- ❑ **Training Node:** Train model weight
 - ❑ 3D Parallelized GPU clusters
- ❑ **Rollout Node:** Perform rollout
 - ❑ Tensor Parallelized remote worker

System Control

- ❑ **Load Balancer:** Balancing request among rollout workers
- ❑ **Transfer Agent:** Stores/ update model weight



Method | Request Migration

Token Level Offload:

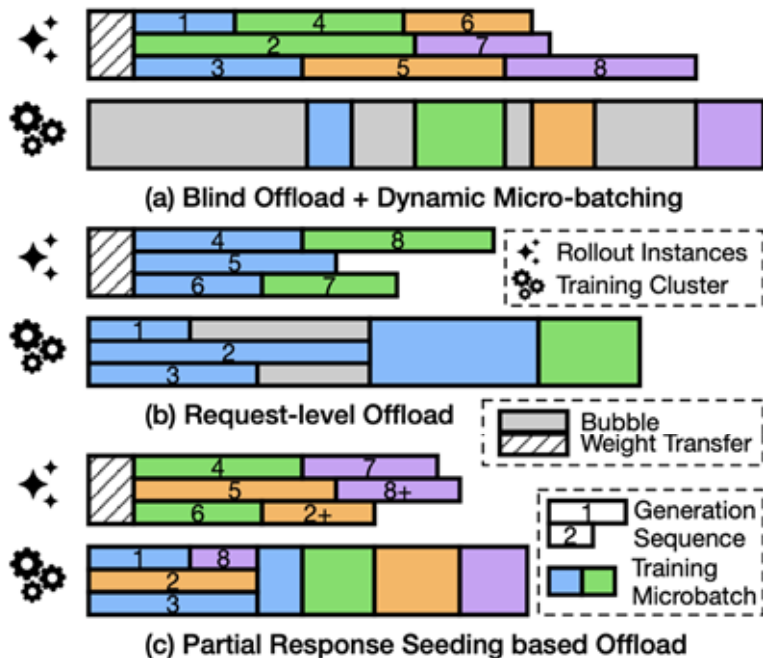
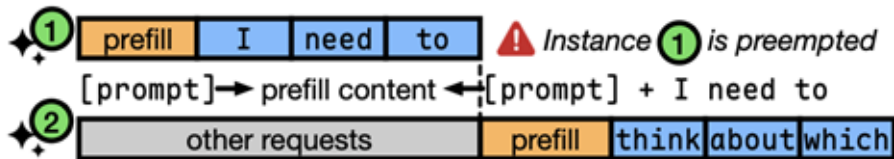
Completed tokens preserved as prefill

→ Better load balancing during rollout

Partial Response Seeding:

Training instance performing rollout when idle

→ Better training cluster utilization



Method | Adaptive Training

Dynamically updated system parameters of RLBoost

T_{seed} : Rollout time on the training cluster

- ❑ If T_{seed} too large $\rightarrow$ rollout instance idel
- ❑ If T_{seed} too small $\rightarrow$ training instance idel

N_{perm} : Maximum rollout instance

- ❑ Instance $> N_{perm} \rightarrow$ no further speed up

Algorithm 1: Adaptive partial response seeding.

Input : N_{resv} : Number of local rollout engines in the reserved training cluster; η : Adaptation rate; T_{init} : Initial seeding window, S : Total number of training steps.

```
1  $\mathcal{M} \leftarrow \emptyset$  // scheduler memory
2  $T_{seed} \leftarrow T_{init}$  // rollout time window on the training cluster

3  $N_{perm} \leftarrow N_{resv}$  // initialize max amount of preemptible
   instances the same as reserved rollout engines

4 for  $s \leftarrow 1$  to  $S$  do
5   EXECUTESTEP( $T_{seed}, N_{perm}$ )
6    $\bar{n}_{prem}, \hat{n}_{prem} \leftarrow \text{MONITORINSTAVAIL}()$ 
7    $t_{train}^{wait}, T_{remote}^{wait} \leftarrow \text{GETIDLETIME}()$ 
8    $t_{train}, t_{remote} \leftarrow \text{GETCOMPUTETIME}()$ 
   // update schedule
9    $T_{seed} \leftarrow T_{seed} + \frac{t_{train}^{wait} - t_{remote}^{wait}}{\eta}$ 
10   $N_{perm} \leftarrow \frac{t_{remote} \bar{n}_{prem} + T_{seed} N_{resv}}{t_{train}}$ 
11  if  $\bar{n}_{prem} = \hat{n}_{prem}$  then
   // memorize schedule optimized under  $\hat{n}_{prem}$ 
12  |  $\mathcal{M}[\hat{n}_{prem}] \leftarrow T_{seed}$ 
13  if  $\hat{n}_{prem} \in \mathcal{M}$  then
   // retrieve latest schedule optimized for  $\hat{n}_{prem}$ 
14  |  $T_{seed} \leftarrow \mathcal{M}[\hat{n}_{prem}]$ 
```

Method | Load Balancer

Select Instance

- ❑ Join the Shortest Queue (JSQ)
- ❑ Delayed Dispatch

Continuous LB

- ❑ Pending Request
 - Migrate 1 request
- ❑ Idle Device
 - Migrate r executing request

Algorithm 2: RLBoost's load balancer.

Input : I : Set of rollout instances; $\mathcal{P}$: Inference batching profile table; Θ : Maximum pending requests threshold.

```
1 function SELECTINSTANCE( $I$ )
2   while true do
3      $C \leftarrow \emptyset$ 
4     foreach  $i \in I$  do
5        $m_i^{\text{pending}} \leftarrow \text{QUERYPENDING}(i)$ 
6       if  $m_i^{\text{pending}} < \Theta$  then
7          $C \leftarrow C \cup \{i\}$ 
8     if  $C \neq \emptyset$  then
9        $i \leftarrow \arg \min_{i \in I} m_i^{\text{pending}}$ 
10      return  $i$ 
11    else
12      WAITANYCOMPLETION()
13 procedure CONTINUOUSLB( $I, \mathcal{P}$ )
14   while true do
15     foreach  $i \in I$  do
16        $m_i^{\text{pending}} \leftarrow \text{QUERYPENDING}(i)$ 
17        $m_i^{\text{exec}} \leftarrow \text{QUERYEXECUTING}(i)$ 
18       if  $\exists i, m_i^{\text{pending}} = 0$  and  $\exists k, m_k^{\text{pending}} > 0$  then
19          $j \leftarrow \arg \max_{k \in I} m_k^{\text{pending}}$ 
20         // migrate a single request
21         MIGRATEREQS( $j \rightarrow i, 1$ )
22       else if  $\exists i, m_i^{\text{exec}} = 0$  then
23          $j \leftarrow \arg \max_{k \in I} m_k^{\text{exec}}$ 
24          $B \leftarrow \text{GETBATCHINGPLATEAU}(\mathcal{P})$ 
25          $r \leftarrow \max(m_j^{\text{exec}} - B, 0)$ 
26         // migrate  $r$  requests
27         MIGRATEREQS( $j \rightarrow i, r$ )
```

Method | Pull Based Weight Transfer

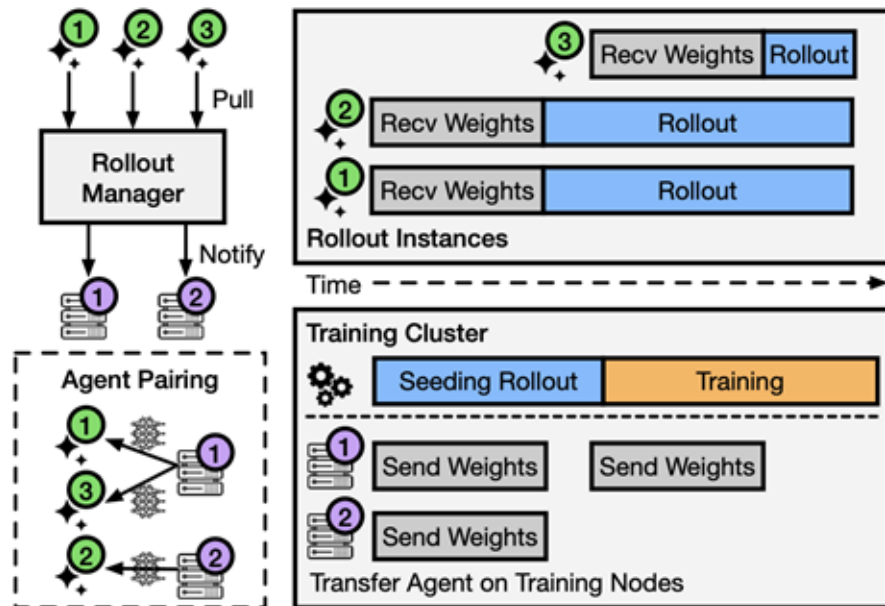
Weight transfer limited by network bandwidth

→ Stand alone weight server

Weight update process } Training Node

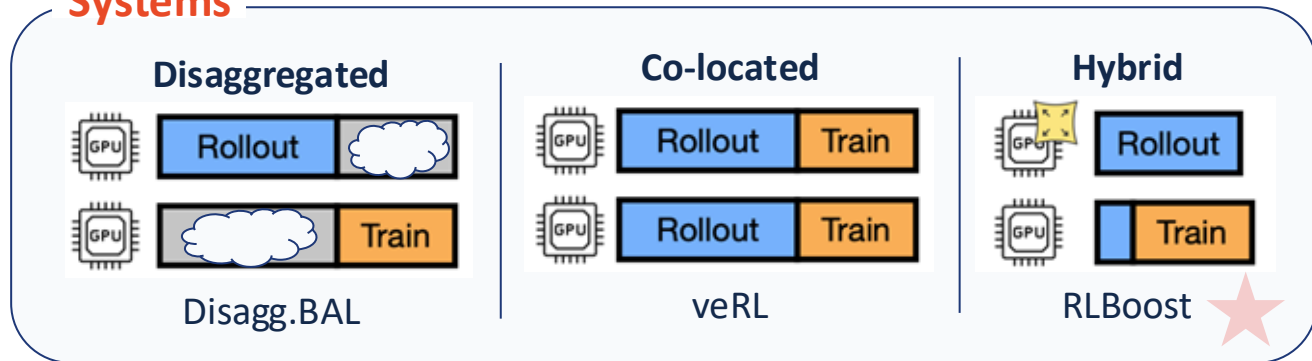
1. Intra cluster all gather
2. Copy weight to server
3. Async pull weight
4. Model inference

} Rollout Node



Evaluation | Setup

Systems

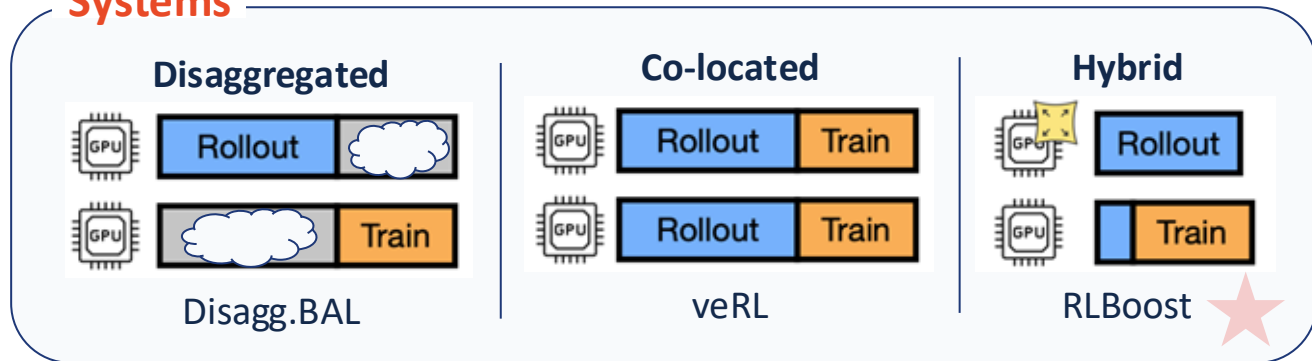


Hardware

Training: **On-demand (reserved)** instances w/ 8 H100 GPUs
Rollout: **Spot (preemptible)** instances w/ 2 H100 GPUs

Evaluation | Setup

Systems



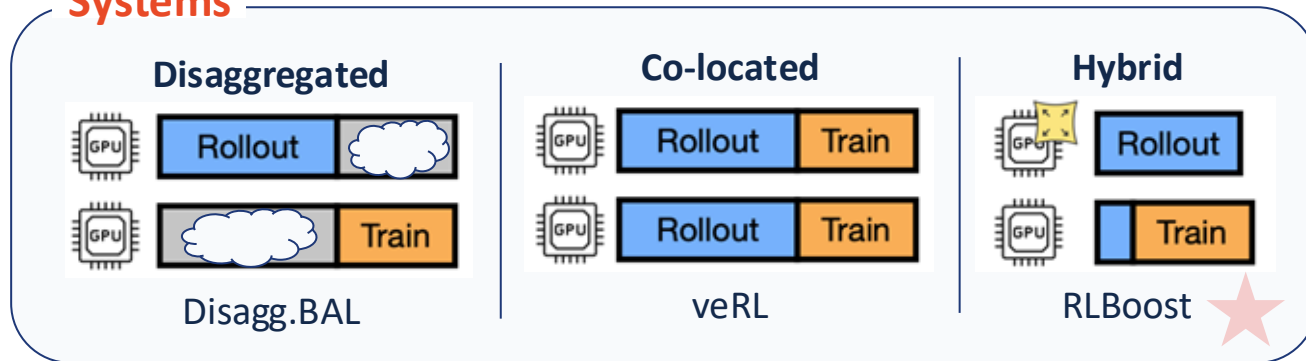
Hardware

Training: On-demand (reserved) instances w/ 8 H100 GPUs

Rollout: Spot (preemptible) instances w/ 2 H100 GPUs

Evaluation | Setup

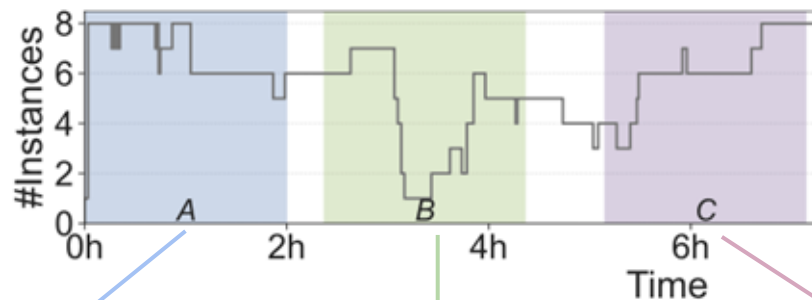
Systems



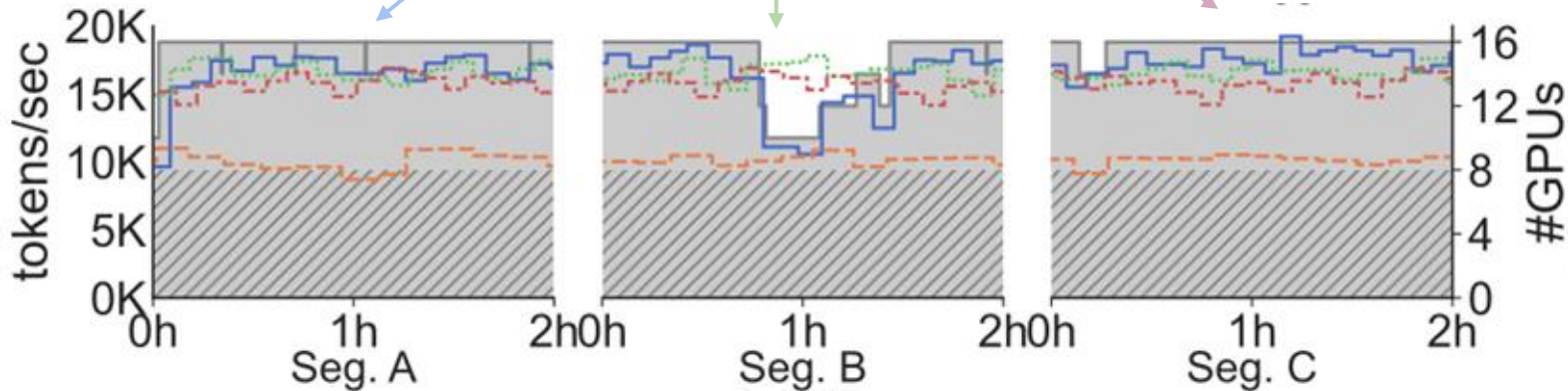
Hardware { **Training: On-demand (reserved)** instances w/ 8 H100 GPUs
Rollout: Spot (preemptible) instances w/ 2 H100 GPUs

Evaluation | Throughput on Qwen3-8B

Spot Instances
Availability



Training
Throughput



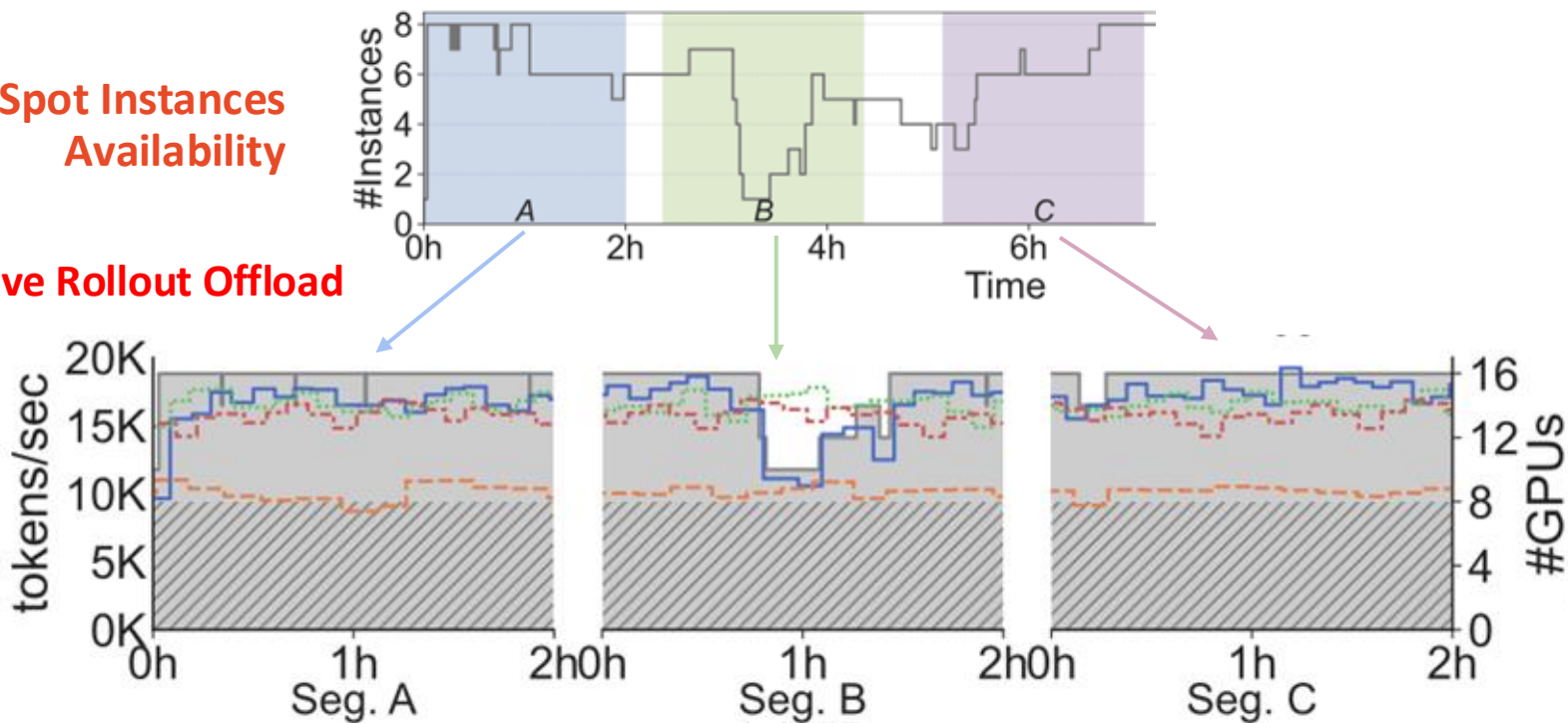
— RLBoost — veRL Disagg.BAL veRL.2x Reserved GPUs Preemptible GPUs

Evaluation | Throughput on Qwen3-8B

Spot Instances
Availability

★ Adaptive Rollout Offload

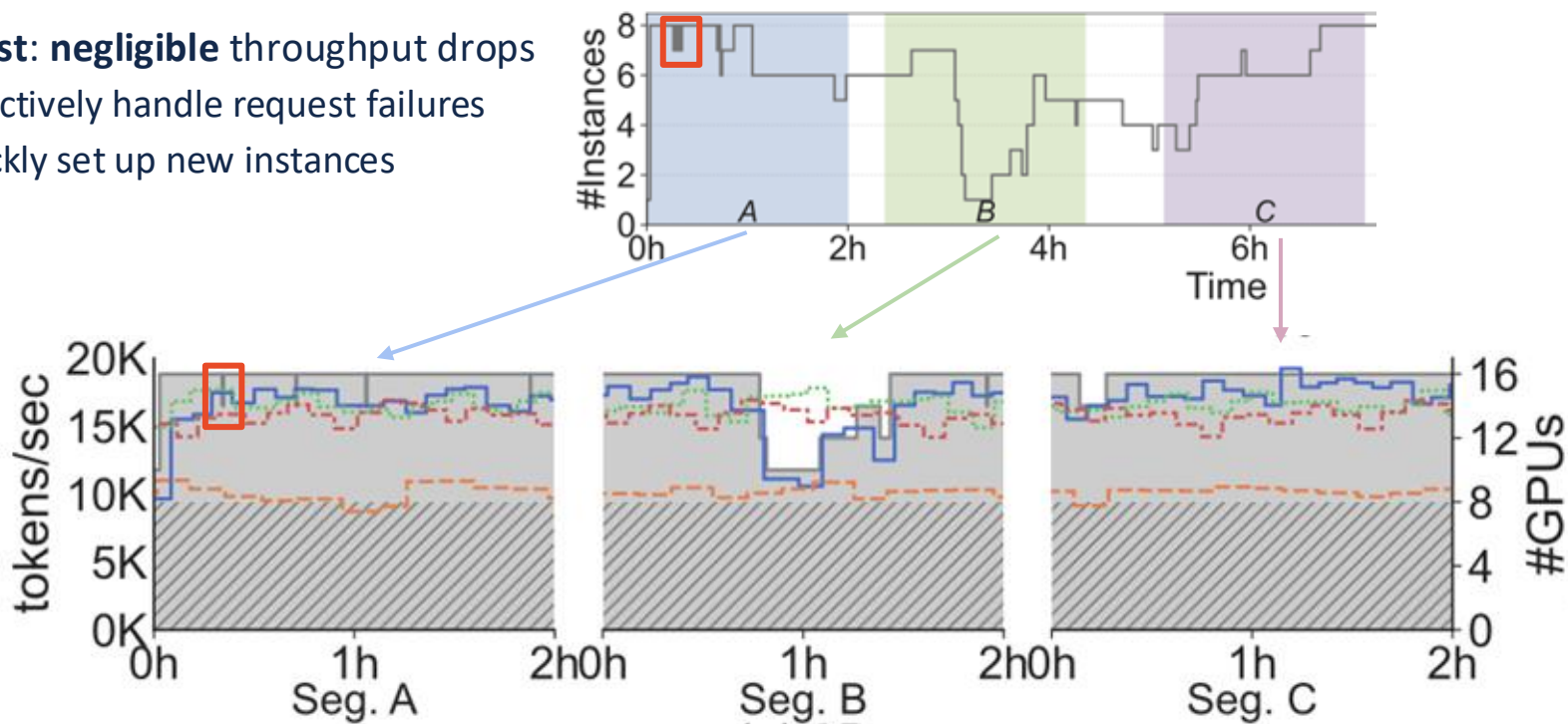
Training
Throughput



Evaluation | Throughput on Qwen3-8B

- **RLBoost: negligible** throughput drops
 - effectively handle request failures
 - quickly set up new instances

Tiny Spikes

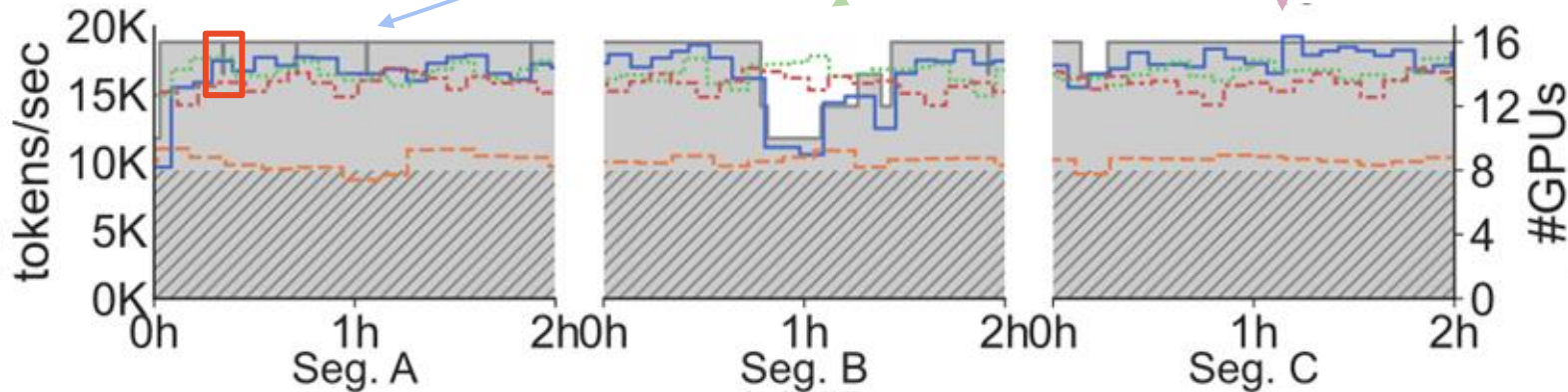
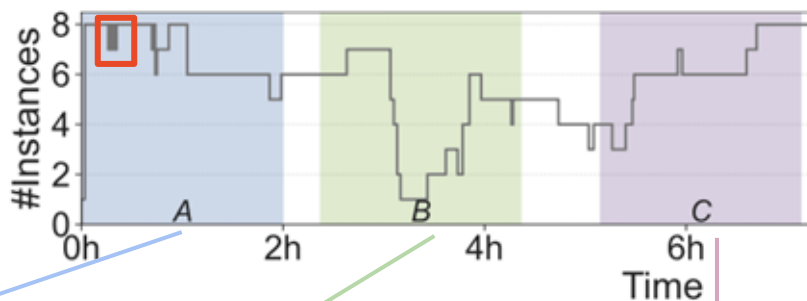


Evaluation | Throughput on Qwen3-8B

- **RLBoost: negligible** throughput drops
 - effectively handle request failures
 - quickly set up new instances

★ **Token-level
Collection**

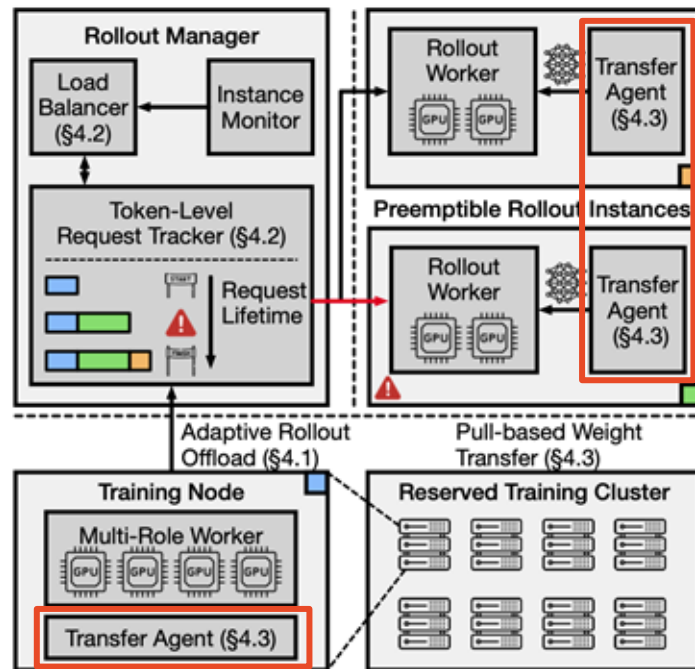
Tiny Spikes



Evaluation | Throughput on Qwen3-8B

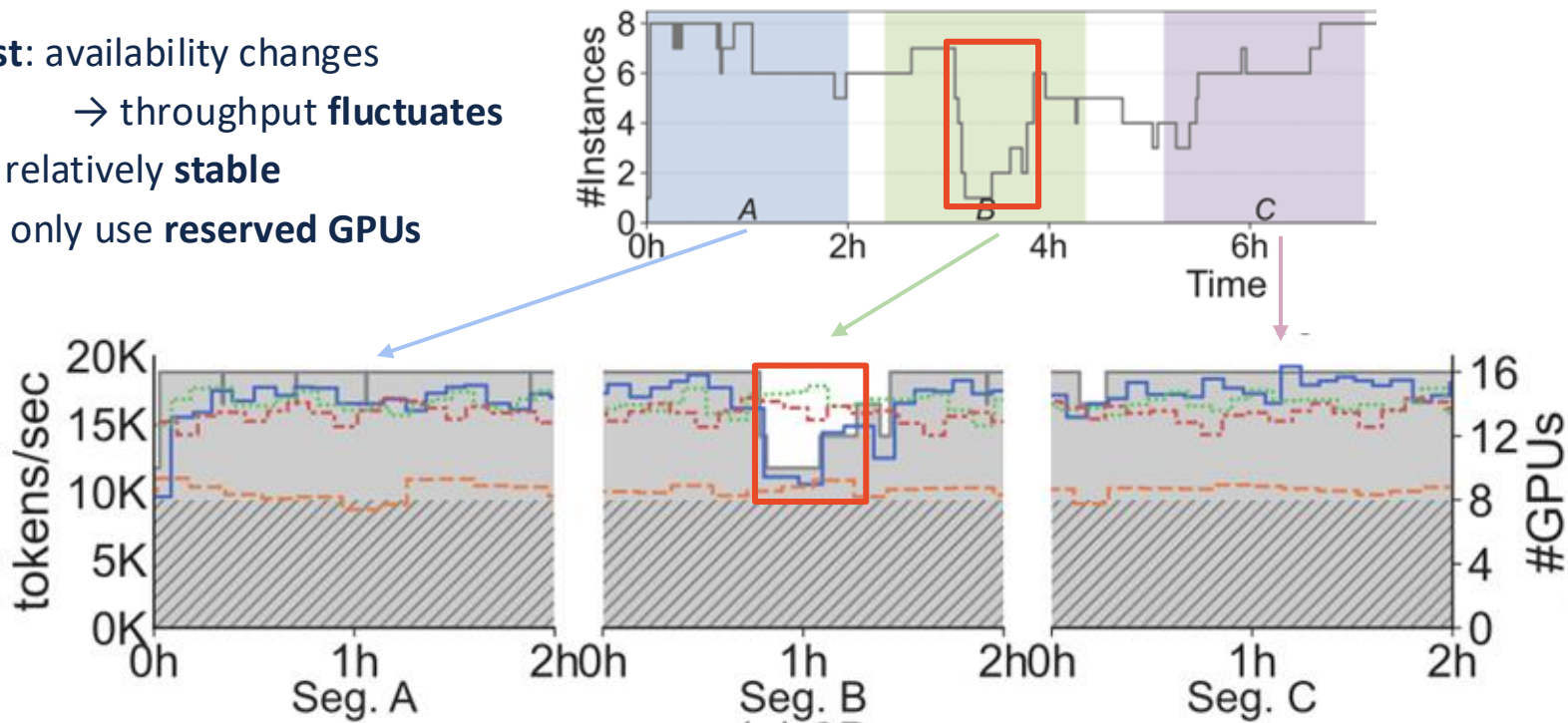
- ❑ **RLBoost: negligible** throughput drops
 - effectively handle request failures
 - quickly set up new instances

★ **Pull-based Weight Transfer**



Evaluation | Throughput on Qwen3-8B

- ❑ **RLBoost**: availability changes
→ throughput **fluctuates**
- ❑ **Other**: relatively **stable**
∴ only use **reserved GPUs**



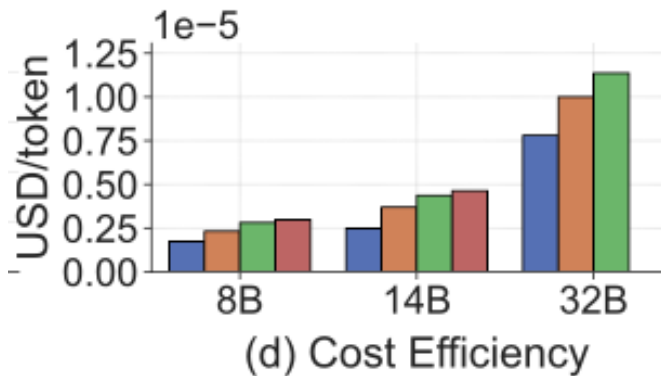
Evaluation | Cost Efficiency

❑ RLBoost vs. veRL

- higher cost savings → **high instance availability (A)**
- lower cost savings → **frequent preemptions + low availability (B)**

❑ Disagg.BAL

- poor cost efficiency → only use **reserved instances**



Future Work

❑ Heterogeneous Hardware Support

- rollout instances operate **independently**
- can be configured with **different** accelerators & parallelism strategies
- **adaptive rollout offload & load balancer**

❑ Weight Transfer Optimization

- each rollout instance **fetches weights directly** from training node
- Future work:
 - (1) **Dynamic Broadcast Tree** - subset fetches from training, others receive from peers
 - (2) **Weight Compression**